

COMS 4720 HW2

1a) We know that when we place any queen at any column say k , we know that each of the already existing queens block at most 3 squares on this column k , which brings us to a conclusion that we have got a lower bound on the number of safe squares that are left in column k :

$$\text{safe}(k) \geq n - 3(k-1)$$

Next, using this formula we can calculate the number of safe squares for each column by substituting the column number with k :

For $k = 1$: $n - 3(1-1) = n$ safe spots

For $k = 2$: $n - 3(2-1) = (n - 3)$ safe spots

For $k = 3$: $n - 3(3-1) = (n - 6)$ safe spots

For $k = 4$: $n - 3(4-1) = (n-9)$ safe spots

.
.
.
.

Note: It goes on until the the number of safe spots become less than or equal to 0.

Now, let us try and calculate how many columns actually have a positive lower bound, so that we can prove that the statement given in the question is correct. Having a positive lower bound means that $\text{safe}(k)$ is always going to be greater than or equal to 1.

$$n - 3(k-1) \geq 1$$

$$n - 3k + 3 \geq 1$$

$$n + 2 \geq 3k$$

$$(n+2)/3 = k$$

Now, we know that when divide anything by 3, we are going to remainder of 0,1 and 2, so that means that we may have to categorize all the safe spots on the basis of what remainder we get after dividing the number by 3. For instance in our case we can do the following:

Group 1(spots with remainder 0): $n, n-3, n-6, n-9, \dots$

Group 2 (spots with remainder 1): $n-1, n-4, n-7, n-10, \dots$

Group 3(spots with remainder 2): $n-2, n-5, n-8, n-11, \dots$

Now, let us multiply all the numbers from all the groups that we just found:

$$n(n-1)(n-2)(n-3)(n-4)(n-5)(n-6) \dots = n!$$

Since $n!$ is distributed equally amongst all of these groups (as each one of them is going to have the largest number as n^m), we can do the following calculation:

$$n^m * n^m * n^m = n^{3m}$$

$$n^{3m} = n!$$

$$n^m = (n!)^{1/3}$$

Hence, we can say that the state space has atleast $(n!)^{1/3}$.

$$b) (n!)^{1/3} \leq 200,000 \times 10^{12}$$

$$(n!)^{1/3} \leq 2 \times 10^{17}$$

$$(n!) \leq (2 * 10^{17})^3$$

$$(n!) \leq 8 * 10^{51}$$

By doing some mathematical calculations we can say that n will roughly be equal to 42 since I found out that $43!$ was more than 200,000 trillion, so that means $n = 42$ in this question.

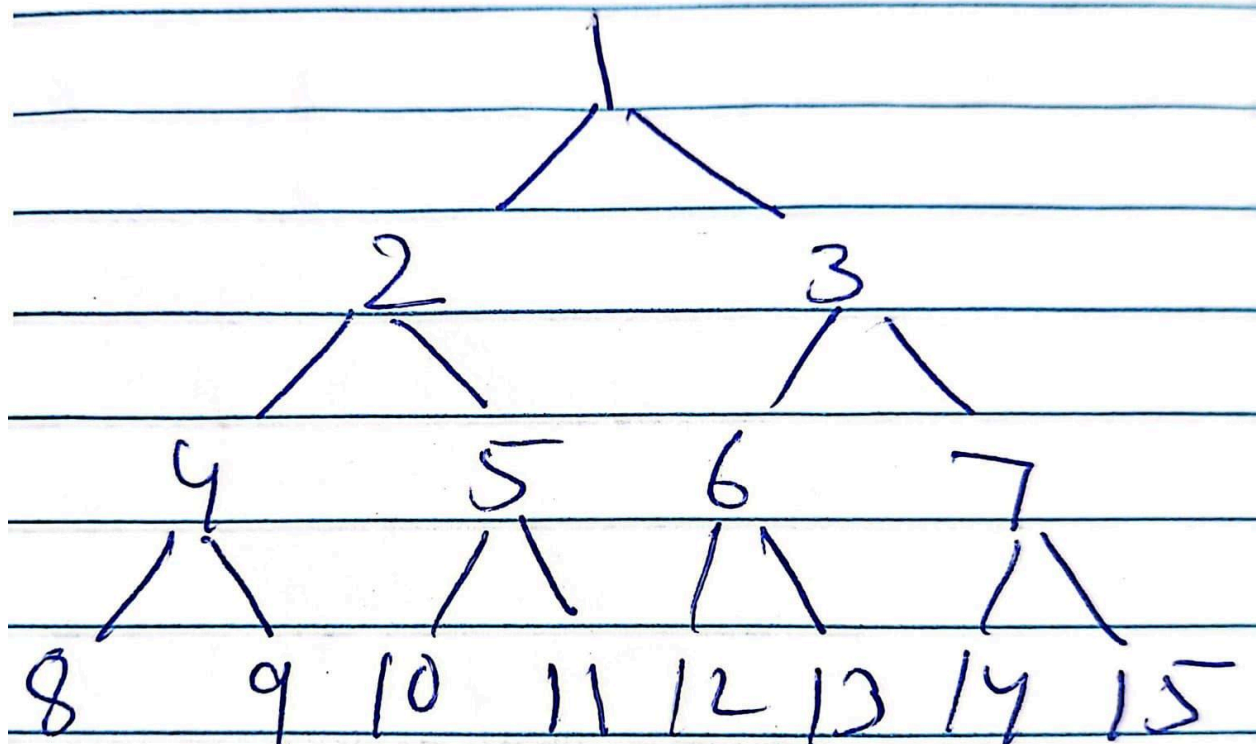
2a) Since DFS can hit its shallow goal immediately, that is, after expanding very few nodes and the A* search, even with an admissible h , tends to expand all the nodes until it reaches the optimal solution, that is $f(n) = f^*$, we can say that the A* search expands more nodes as compared to DFS and hence the given statement is **false**.

b) Since $h(n) = 0$ does not overestimate anything (as it assumes that there is no cost left to the goal), we can say that the statement is **true**.

c) We can say that this statement is **true** because BFS does not depend on the cost as it explores by depth, which means that it wouldn't be impacted by the zero step costs, as the depth levels may still continue to grow until BFS reaches its goal.

d) Since the Manhattan distance calculates the distance in terms of one square at a time and not in terms of the move, plus the Manhattan distance assumes that the chessboard is empty and in our case it is not, we can say that this statement is **false**. I can also prove this through an example, let us assume that we have the rook at position (1,1) and it wants to move down to (7,7), the Manhattan distance is going to be 7, however if nothing was in front of the rook it could just do it in 1 step or even if it had something in between in its path, it still won't 7 moves to move to (7,7), if we think realistically, hence we can say that in most of the cases the Manhattan distance overestimates the true cost and hence it is not admissible heuristic for this problem.

3a)



b) **BFS:** 1 2 3 4 5 6 7 8 9 10 11

Depth Limit Search with limit 3: 1 2 4 8 9 5 10 11

Iterative Deepening search:

Iteration 0: 1

Iteration 1: 1 2 3

Iteration 2: 1 2 4 5 3 6 7

Iteration 3: 1 2 4 8 9 5 10 11

4) Let us consider an example of the following state space:

State space: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow \dots \rightarrow m \rightarrow n \rightarrow o$

Goal State: m

DFS: It expands each node once, so that's why its time complexity is $O(V+E)$, where $V(\text{vertices}) = n$ (number of elements) + 1 and $E(\text{number of edges}) = n + 1$, so we can say that the time complexity of DFS is $O(n)$.

Iterative Deepening search: Since it runs with depth-limited DFS repeatedly, with limits such as 1,2,3,...,n, we say that it expands in the following way:

$$1+2+3+\dots+(n-1) = O(n^2)$$

Since $O(n^2) > O(n)$, we can say that in our example, the iterative depth-first search performs much worse than the depth-first search.